

WAB

Provadis School of International Management and Technology

Examining the Feasibility of Machine Learning-Based LogP Prediction on an ESP32 Microcontroller

A Proof-of-Concept Implementation and Performance Analysis

Rubin Chempananickal James

rubin.chempananickal-james@stud-provadis-hochschule.de

Matriculation Number: D876

Department: Information Technology

Module: Embedded Systems and Software

Reviewer: Prof. Dr. Eric Hutter

March 30, 2026

Abstract

Abstrakt

Contents

Abstract	II
1 Introduction	1
1.1 Background	1
1.2 Motivation	1
1.3 Research Question	3
1.4 Previous Work	3
2 Methods	4
2.1 Dataset	4
2.2 Design	4
2.3 ESP32 Deployment	5
3 Results	7
4 Discussion	9
5 Conclusion	10
Bibliography	i
AI Declaration	iv
Declaration of Authorship	vii

List of Figures

1.1	A QR Code encoding the ECFP4 fingerprint of aspirin. The SMILES string was initially canonicalized via RDKit ¹⁶ , and the fingerprint was generated using the Morgan algorithm ¹⁷ with a radius of 2 and a bit vector length of 2048. The resulting bit vector was converted to a byte stream then encoded into a QR code using the qrcode library in Python.	2
2.1	Training and validation loss and MAE curves over the 12 epochs.	5
2.2	Diagram showing the stages of canonicalization, fingerprint generation, neural network training, quantization, and ESP32 deployment.	6
3.1	Scatter plot of true vs. predicted logP values for the test set. The diagonal line represents perfect predictions. The "plateau" of predictions at the top right is due to a clamping effect caused by the quantization process. . . .	7
3.2	Histogram of absolute errors between predicted and true logP values. The majority of predictions have an absolute error below 0.5, with a long tail extending to higher errors.	8
3.3	Citrate ion, the molecule with the worst prediction error (4.94). The ground truth logP is -6.03, while the predicted logP is -1.09.	8

List of Tables

3.1	Performance metrics on the test set.	7
3.2	Percentage of predictions within certain absolute error thresholds.	8

Glossary

Adaptive Moment Estimation (Adam) An optimization algorithm used in training machine learning models that computes adaptive learning rates for each parameter. 4

ESP32 Espressif Systems' low-cost, low-power system on a chip (SoC) with integrated Wi-Fi and Bluetooth capabilities. 3, 5

Extreme Gradient Boosting (XGBoost) An ensemble learning method that uses gradient boosting to combine the predictions of multiple weak learners (usually decision trees) to create a stronger predictive model. 3

fingerprint A binary vector representation of a molecule, where each bit represents the presence or absence of a particular substructure or feature in the molecule. 2, 3

lipophilicity The chemical property of being fat-soluble, often quantified by the octanol-water partition coefficient ($\log P$). 1

$\log P$ Logarithm of the octanol-water partition coefficient. A measure of a compound's hydrophobicity, or how well it dissolves in fats versus water. 1–4, 6–8, V

Multi-layer Perceptron (MLP) A feed-forward artificial neural network composed of an input layer, one or more fully connected hidden layers, and an output layer. 4

QR code Quick Response Code, a type of matrix barcode (or two-dimensional barcode) that can store information such as a URL, text, or other data. 2

Random Forest (RF) An ensemble learning method for classification and regression tasks that constructs multiple bootstrapped decision trees during training. 3

Rectified Linear Unit (ReLU) A type of activation function used in neural networks that outputs the input directly if it is positive, otherwise, it outputs zero. 4

Simplified Molecular Input Line Entry System (SMILES) A notation that allows a user to represent a chemical structure as a string of ASCII characters. 2–4, V

TensorFlow Lite (TFLite) A framework for deploying machine learning models on mobile and embedded devices. Currently being rebranded as LiteRT. 5

ZINC20 ZINC Is Not Commercial (2020), a free database of commercially available compounds for virtual screening. 3, 4

1 Introduction

1.1 Background

The octanol-water partition coefficient ($\log P$) measures how well a compound dissolves in octanol versus water. It is a fundamental physicochemical property that quantifies the lipophilicity of a molecule.

The logarithm of partition coefficient ($\log P$) between n-octanol and water is a critical property related to the physicochemical and physiological properties of a drug molecule, such as absorption, distribution, metabolism, excretion and toxicity (ADMET) and target protein binding^{1,2,3,4}. Thus it is frequently used in describing the druggability of a pharmaceutical molecule. The first $\log P$ calculation method was developed by Fujita et al. in 1960s^{5,6}. Early on, the structural information of receptor is not easy to obtain, medicinal chemists frequently used quantitative structure-activity relationship (QSAR) to correlate the binding energy of a ligand with its physicochemical characteristics, and $\log P$ was regarded as a key descriptor measuring a drug’s hydrophobicity. Nowadays, Lipinski’s rule of five^{7,8}, published in the 1990s, has been widely used as simple rules to determine the druglikeness of a pharmaceutical molecule. $\log P$ smaller than 5 is one of the four rules in Lipinski’s rule of 5.⁹

1.2 Motivation

To date, there have been many computational methods developed to predict $\log P$ values, including aggregating the $\log p$ contributions of individual fragments^{10,11}, free energy based calculations⁹, and more recently, employing machine learning based methods to predict $\log P$ values based on a precomputed set of input features^{12,13,14}.

Fragment based $\log P$ calculations involve having a lookup table of individual fragments, and a method of constructing a molecular graph and finding the fragments within it. Both this and the free energy based calculations require a non-trivial amount of computational resources, and are therefore infeasible to be implemented as a near real-time prediction method on a microcontroller.

The machine learning based approaches could potentially be feasible, if a neural network could be trained to predict $\log P$ values and translated into a microcontroller compatible format using LiteRT¹⁵ (formerly TensorFlow Lite).

The reason for exploring the ability to predict $\log P$ on a microcontroller is to enable real-time, on-site predictions in various applications, such as environmental monitoring or portable drug discovery tools. By offloading the prediction task away from a host computer and onto a microcontroller, one can potentially reduce latency in systems that require immediate response, such as a microfluidic dispensing device that needs to make decisions based on the $\log P$ of a compound in real-time.

Since the molecular fingerprint is a bit vector that could be encoded as a QR code (see Figure 1.1 for an example), the microcontroller could also be used in scenarios where a user can scan said code printed on the sample in a laboratory or field setting with a portable device and receive an immediate prediction of the $\log P$ value, and if this approach seems feasible and accurate enough, it could be extended to predicting other properties as well, without needing to access a cloud-based service or a powerful computer.



Figure 1.1: A QR Code encoding the ECFP4 fingerprint of aspirin. The SMILES string was initially canonicalized via RDKit¹⁶, and the fingerprint was generated using the Morgan algorithm¹⁷ with a radius of 2 and a bit vector length of 2048. The resulting bit vector was converted to a byte stream then encoded into a QR code using the qrcode library in Python.

Also, by decoupling the fingerprint generation (a deterministic process on canonical SMILES strings) and the prediction (a non-deterministic process owing to the nature of machine learning models), and by binding the prediction to a specific piece of hardware (a "black box", as it were), one can make the entire system more modular and easier to debug. As an added bonus, the host computer wouldn't need to have any machine learning libraries installed.

1.3 Research Question

The main research question of this project is: How accurately can a quantized machine learning model predict logP values based on its ECFP4 fingerprint when deployed on an ESP32 microcontroller?

1.4 Previous Work

To date, there have been multiple efforts to train machine learning models to serve as logP predictors, such as the work by Ognichenko et.al.¹² who trained an RF model on the 2D Simplex Representation of Molecular Structures (SiRMS) descriptors¹⁸ of 10973 compounds, and the work by Baikété et.al.¹³ who trained a variety of machine learning models (RF, XGBoost, among others) on 623 molecular descriptors generated by the RDKit¹⁶ and Mordred¹⁹ packages for a dataset of 1,117 compounds. As of the time of writing, there have been no known efforts to deploy a machine learning based logP predictor on a microcontroller.

This current project aims to build upon the efforts of Gómez-Bombarelli et.al.²⁰, who created a neural network based logP predictor on a subset of the ZINC20 dataset²¹ as part of creating a variational autoencoder for molecular generation. Their dataset serves as the basis for training the model in this project, as it provides a large enough set of drug-like compounds with valid SMILES strings.

2 Methods

2.1 Dataset

The base dataset used in this project is a subset of the ZINC20 dataset²¹, which contains 249,455 commercially available drug-like molecules and their corresponding logP values. Specifically, this subset was extracted at random from the entire ZINC20 dataset by Gómez-Bombarelli et al.²⁰.

The logP values within the ZINC20 dataset were calculated using Molinspiration’s MiLogP algorithm, which calculates logP based on group contributions²². Due to the limited availability of large datasets of experimentally confirmed logP values for drug-like molecules, and due to the limited scope of this project, this computed logP will be treated as the ground truth for the purposes of training and evaluating the machine learning model.

2.2 Design

Python²³ version 3.12.13 served as the primary programming language for this project. The entire project was conducted on a laptop running Windows 11 25H2.

The dataset was randomly split into training (80%), validation (10%), and test (10%) sets, with the random seed set to 42 to ensure reproducibility.

The input features for the model are 2048-bit Morgan fingerprints (radius 2) generated from the SMILES representations of molecules. The SMILES were canonicalized and the fingerprint bit vectors were generated using the RDKit¹⁶ library in Python.

The machine learning model is a MLP with a single hidden layer of 128 units, using a ReLU activation function, followed by a linear output layer that predicts the logP value. Keras²⁴ was used as the high-level API for building and training the neural network, while TensorFlow²⁵ served as the backend. Model training was performed fully on the CPU using the Adam optimizer²⁶ and mean squared error (MSE) as the loss function.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (2.1)$$

where y_i is the true logP value and $\hat{y}_i$ is the predicted logP value for the i -th sample.

Early stopping was employed to prevent overfitting, monitoring the validation loss and restoring the best weights. This meant that with a batch size of 256 and the TensorFlow random seed set to 42, the training converged after 12 out of a maximum of 20 epochs (see Figure 2.1).

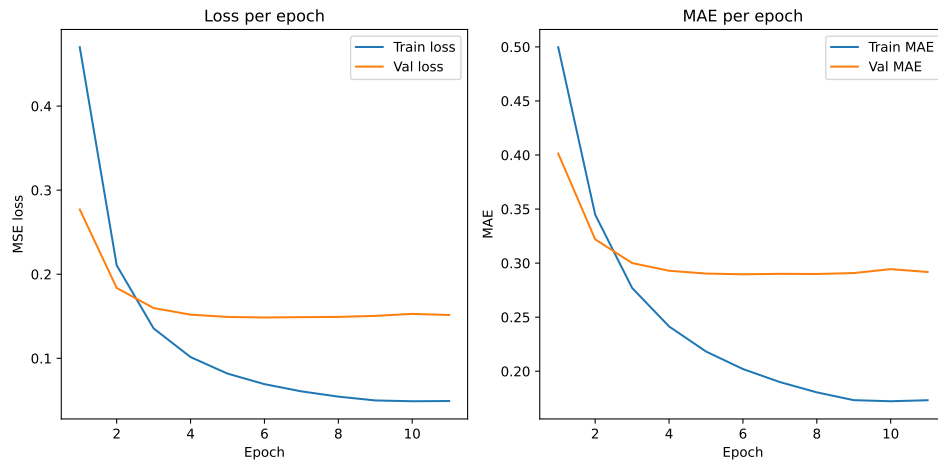


Figure 2.1: Training and validation loss and MAE curves over the 12 epochs.

2.3 ESP32 Deployment

To reduce the model size, the final model was quantized to 8-bit integers using TFLite’s post-training quantization method. The resulting flatbuffer was saved as a .tflite file.

This .tflite file was then converted to a C array using a python tool that emulates the POSIX ‘xxd’ command, and the resulting .cc and .h files, along with an API script, were built and flashed onto an ESP32 microcontroller using ESP-IDF version 5.5.3.

All the necessary code files, along with Visual Studio Code specific instructions for setting up the ESP-IDF environment, building the project, and flashing the firmware, can be found in the supplementary materials.

ESP32 specifications²⁷:

- Model: ESP32-WROOM-32D
- CPU: Dual-core Xtensa LX6 microprocessor, up to 240 MHz
- RAM: 520 KB SRAM
- Flash Storage: 4 MB

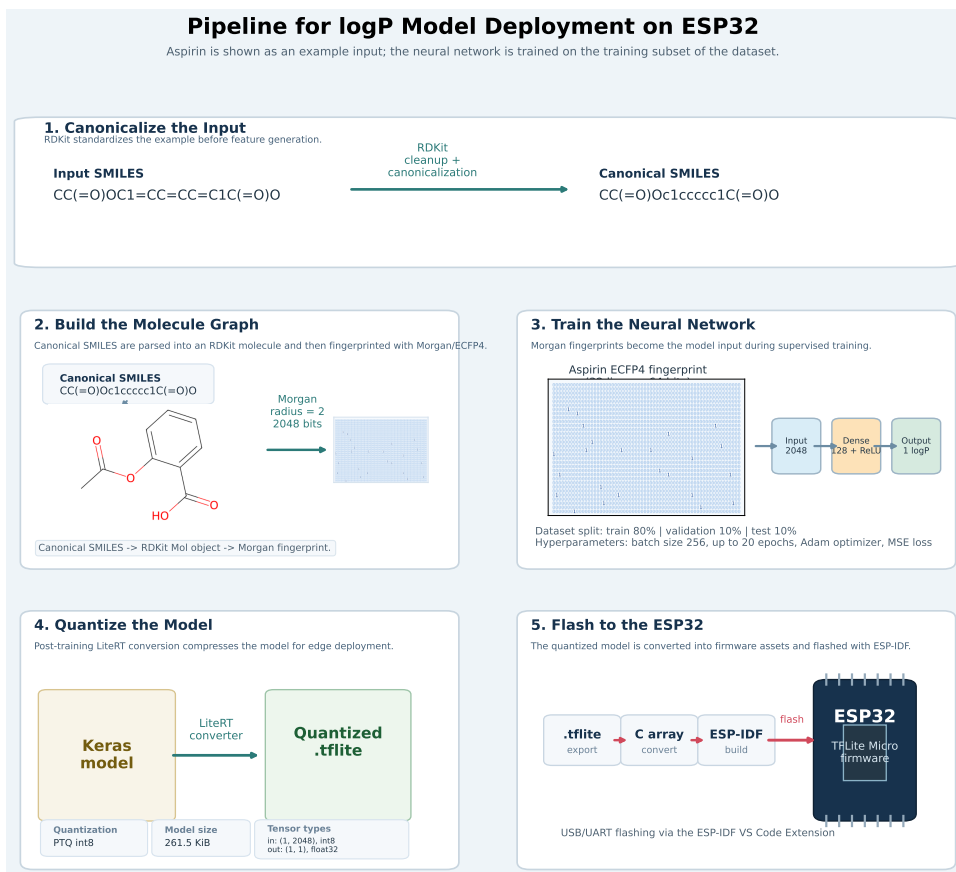


Figure 2.2: Diagram showing the stages of canonicalization, fingerprint generation, neural network training, quantization, and ESP32 deployment.

The accuracy of the model was then tested on the test set of 24,946 compounds by running inference on the ESP32 and comparing the predicted logP values against the ground truth. The measured performance metrics include mean absolute error (MAE), root mean squared error (RMSE), the coefficient of determination (R^2), and the Pearson correlation coefficient (r).

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (2.2)$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (2.3)$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (2.4)$$

$$r = \frac{\sum_{i=1}^n (y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}})}{\sqrt{\sum_{i=1}^n (y_i - \bar{y})^2 \sum_{i=1}^n (\hat{y}_i - \bar{\hat{y}})^2}} \quad (2.5)$$

where n is the number of samples, y_i is the true logP value, $\hat{y}_i$ is the predicted logP value, $\bar{y}$ is the mean of the true logP values, and $\bar{\hat{y}}$ is the mean of the predicted logP values.

3 Results

Metric	Value
Number of Samples	24,946
Successfully Predicted	100%
Mean Absolute Error (MAE)	0.29
Root Mean Squared Error (RMSE)	0.39
Median Absolute Error	0.23
Maximum Absolute Error	4.94
R^2	0.92
Pearson r	0.96
Mean Inference Time	53.13 ms
Mean Round Trip Time	237.97 ms

Table 3.1: Performance metrics on the test set.

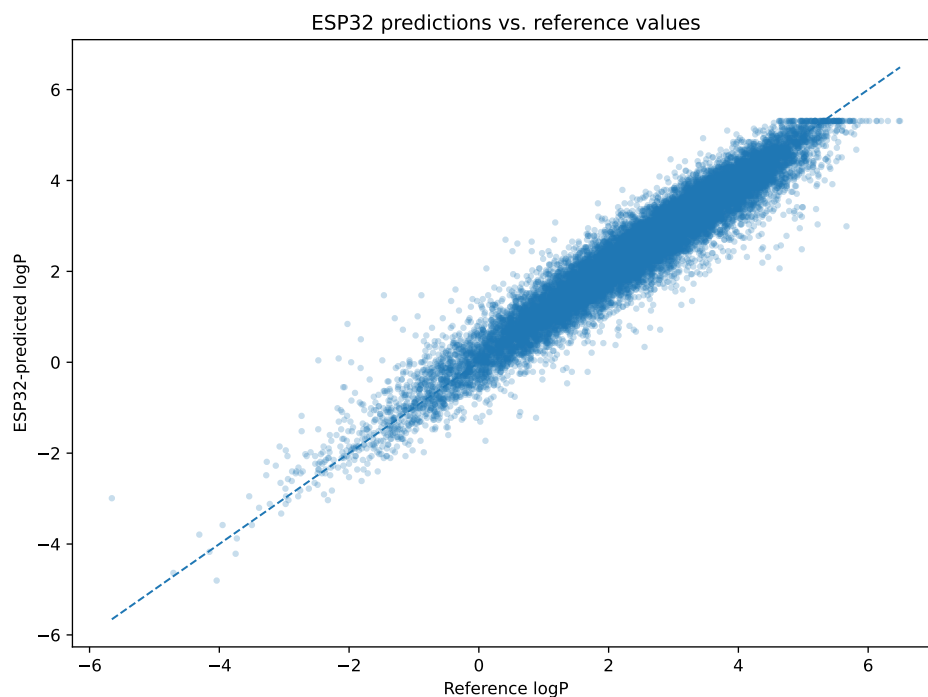


Figure 3.1: Scatter plot of true vs. predicted logP values for the test set. The diagonal line represents perfect predictions. The "plateau" of predictions at the top right is due to a clamping effect caused by the quantization process.

Metric	Value
Within 0.25 logP units	53.12%
Within 0.50 logP units	83.10%
Within 1.00 logP units	98.06%

Table 3.2: Percentage of predictions within certain absolute error thresholds.

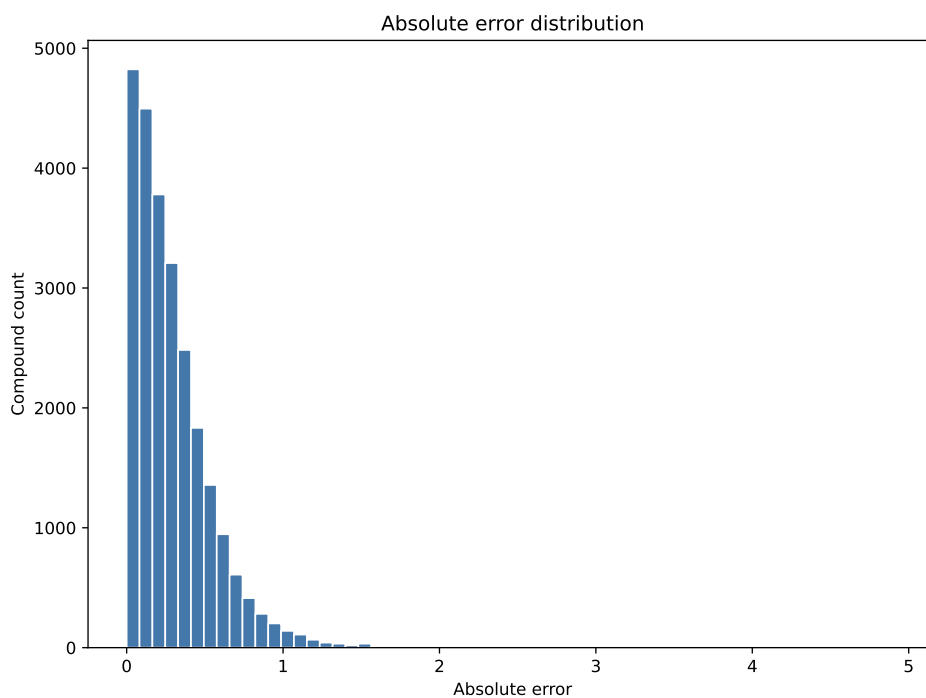


Figure 3.2: Histogram of absolute errors between predicted and true logP values. The majority of predictions have an absolute error below 0.5, with a long tail extending to higher errors.

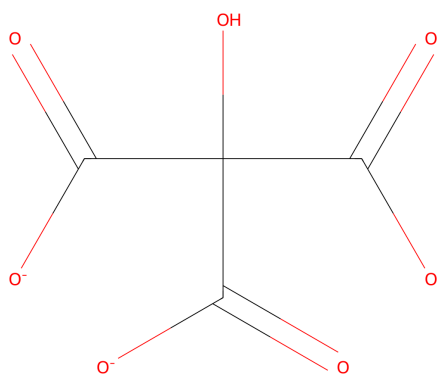


Figure 3.3: Citrate ion, the molecule with the worst prediction error (4.94). The ground truth logP is -6.03, while the predicted logP is -1.09.

4 Discussion

5 Conclusion

Bibliography

- [1] Jason D. Hughes et al. „Physiochemical drug properties associated with in vivo toxicological outcomes“. In: *Bioorganic & Medicinal Chemistry Letters* 18.17 (2008), pp. 4872–4875. ISSN: 0960-894X. DOI: <https://doi.org/10.1016/j.bmcl.2008.07.071>. URL: <https://www.sciencedirect.com/science/article/pii/S0960894X08008500>.
- [2] Michael J Waring. „Defining optimum lipophilicity and molecular weight ranges for drug candidates—molecular weight dependent lower log D limits based on permeability“. In: *Bioorganic & Medicinal Chemistry Letters* 19.10 (2009), pp. 2844–2851.
- [3] M Paul Gleeson. „Generation of a set of simple, interpretable ADMET rules of thumb“. In: *Journal of Medicinal Chemistry* 51.4 (2008), pp. 817–834.
- [4] Ted W Johnson, Klaus R Dress, and Martin Edwards. „Using the Golden Triangle to optimize clearance and oral absorption“. In: *Bioorganic & Medicinal Chemistry Letters* 19.19 (2009), pp. 5560–5564.
- [5] Toshio Fujita, Junkichi Iwasa, and Corwin Hansch. „A new substituent constant, π , derived from partition coefficients“. In: *Journal of the American Chemical Society* 86.23 (1964), pp. 5175–5180.
- [6] Junkichi Iwasa, Toshio Fujita, and Corwin Hansch. „Substituent constants for aliphatic functions obtained from partition coefficients“. In: *Journal of Medicinal Chemistry* 8.2 (1965), pp. 150–153.
- [7] CA Lipinski et al. „Polar molecular surface properties predict the intestinal absorption of drugs in humans“. In: *Adv. Drug Delivery Rev.* 46.1–3 (2001), pp. 3–26.
- [8] Christopher A Lipinski. „Lead-and drug-like compounds: the rule-of-five revolution“. In: *Drug discovery today: Technologies* 1.4 (2004), pp. 337–341.
- [9] Yuchen Sun et al. „Development and test of highly accurate endpoint free energy methods. 2: Prediction of logarithm of n-octanol-water partition coefficient (logP) for druglike molecules using MM-PBSA method“. In: *Journal of Computational Chemistry* 44.13 (2023), pp. 1300–1311. DOI: <https://doi.org/10.1002/jcc.27086>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/jcc.27086>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/jcc.27086>.
- [10] Roelof F Rekker and Raimund Mannhold. „Calculation of drug lipophilicity: the hydrophobic fragmental constant approach“. In: *(No Title)* (1992).

- [11] Corwin Hansch et al. „Substituent constants for correlation analysis“. In: *Journal of medicinal chemistry* 20.2 (1977), pp. 304–306.
- [12] Liudmyla N. Ognichenko et al. „QSPR Prediction of Lipophilicity for Organic Compounds Using Random Forest Technique on the Basis of Simplex Representation of Molecular Structure“. In: *Molecular Informatics* 31.3-4 (2012), pp. 273–280. DOI: <https://doi.org/10.1002/minf.201100102>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/minf.201100102>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/minf.201100102>.
- [13] Juda Baikété, Alhadji Malloum, and Jeanet Conradie. „Comparative study of machine learning methods for accurate prediction of logP and pKb“. In: *Artificial Intelligence Chemistry* 4.1 (2026), p. 100101. ISSN: 2949-7477. DOI: <https://doi.org/10.1016/j.aichem.2025.100101>. URL: <https://www.sciencedirect.com/science/article/pii/S2949747725000181>.
- [14] Qingda Zang et al. „In Silico Prediction of Physicochemical Properties of Environmental Chemicals Using Molecular Fingerprints and Machine Learning“. In: *Journal of Chemical Information and Modeling* 57.1 (2017). PMID: 28006899, pp. 36–49. DOI: 10.1021/acs.jcim.6b00625. eprint: <https://doi.org/10.1021/acs.jcim.6b00625>. URL: <https://doi.org/10.1021/acs.jcim.6b00625>.
- [15] *LiteRT is Google’s on-device framework for high-performance ML & GenAI deployment on edge platforms*. URL: <https://ai.google.dev/edge/litert> (visited on 03/29/2026).
- [16] *RDKit: Open-source cheminformatics software*. DOI: <https://doi.org/10.5281/zenodo.591637>. URL: <https://www.rdkit.org/> (visited on 03/29/2026).
- [17] Harry L Morgan. „The generation of a unique machine description for chemical structures-a technique developed at chemical abstracts service.“ In: *Journal of chemical documentation* 5.2 (1965), pp. 107–113.
- [18] Victor E Kuz'min et al. „Hierarchic system of QSAR models (1D–4D) on the base of simplex representation of molecular structure“. In: *Journal of molecular modeling* 11.6 (2005), pp. 457–467.
- [19] Hiroto Moriaki et al. „Mordred: a molecular descriptor calculator“. In: *Journal of cheminformatics* 10.1 (2018), p. 4.
- [20] Rafael Gómez-Bombarelli et al. „Automatic chemical design using a data-driven continuous representation of molecules“. In: *ACS central science* 4.2 (2018), pp. 268–276. DOI: <https://doi.org/10.1021/acscentsci.7b00572>. URL: <https://pubs.acs.org/doi/10.1021/acscentsci.7b00572>.
- [21] John J Irwin et al. „ZINC: a free tool to discover chemistry for biology“. In: *Journal of chemical information and modeling* 52.7 (2012), pp. 1757–1768.

- [22] *Molinspiration: logP - octanol-water partition coefficient*. URL: <https://www.molinspiration.com/services/logp.html> (visited on 03/29/2026).
- [23] Guido Van Rossum and Fred L. Drake. *Python 3 Reference Manual*. Scotts Valley, CA: CreateSpace, 2009. ISBN: 1441412697.
- [24] François Chollet et al. *Keras*. <https://keras.io>. 2015.
- [25] Martín Abadi et al. *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. Software available from tensorflow.org. 2015. URL: <https://www.tensorflow.org/>.
- [26] Diederik Kingma and Jimmy Ba. „Adam: A Method for Stochastic Optimization“. In: *International Conference on Learning Representations* (Dec. 2014).
- [27] *ESP32-WROOM-32D & ESP32-WROOM-32U Datasheet*. URL: https://documentation.espressif.com/esp32-wroom-32d_esp32-wroom-32u_datasheet_en.pdf (visited on 03/29/2026).

AI Declaration

The usage of AI tools within this project is documented here. I solemnly declare that I have documented all interactions with AI tools, including the prompts used and the outputs received.

System	Prompt	Usage
GitHub Copilot 1	Asked to delete the existing project and provide a simple LaTeX template with a shared preamble, a Chapters subdirectory, TOC, glossary, abbreviations, and Roman numerals for non-content pages.	Template structure and LaTeX setup provided
GitHub Copilot 2	Requested additional title page lines (WAB header, reviewer, module) for the main document.	Title page metadata and layout updated
GitHub Copilot 3	Requested uppercase Roman numerals for front matter and lowercase Roman numerals for back matter page numbering.	Page numbering adjusted in main and exposee
GitHub Copilot 4	Requested exposee title page to include shared info fields (WAB header, department, module, reviewer).	Exposee title page updated to include shared metadata
GitHub Copilot 5	Requested adding the provadis-hochschule.pdf logo to the top right corner of both main and exposee title pages.	Logo added to top right of both title pages
GitHub Copilot 6	Reported that Glossary and Abbreviations sections were missing from compiled output.	Added example <code>\newacronym</code> entries to abbreviations.tex so the Abbreviations section displays properly
GitHub Copilot 7	Reported that glossary section still not appearing in final PDF.	Updated settings.tex to include <code>automake</code> option in glossaries package to enable automatic glossary generation
GitHub Copilot 8	Asked for boilerplate to train a TFLite model for ESP32 to estimate logP from SMILES.	Provided a Python training script (RDKit + Keras) with TFLite export
GitHub Copilot 9	Asked how to run inference on an ESP32 with the trained model.	Provided a high-level TFLite Micro deployment workflow for ESP32
GitHub Copilot 10	Asked for a beginner-friendly README in Code/ with step-by-step setup and usage.	Added a novice-oriented Code/README.md with installation, training, and ESP32 notes

System	Prompt	Usage
GitHub Copilot 11	Asked for a TFLite Micro skeleton and beginner ESP32 setup steps.	Added <code>Code/tflite_micro_skeleton.cpp</code> and expanded <code>Code/README.md</code> with ESP32 instructions
GitHub Copilot 12	Reported the TFLite Micro skeleton was incomplete.	Completed the skeleton with quantized support, Arduino setup, and a runnable loop
GitHub Copilot 13	Asked to avoid Roman-numbered pages in glossary references.	Explained how <code>\glsaddall</code> affects page references
GitHub Copilot 14	Asked to replace <code>iterrows</code> in the dataset loader with a better pattern.	Replaced <code>loop</code> with <code>Series.map</code> + boolean mask + <code>np.stack</code> for vectorized fingerprint loading
GitHub Copilot 15	Asked for help to deploy the quantized logP model onto an ESP32 with LiteRT.	Added a dedicated ESP-IDF project, a host-side Python serial script, and usage instructions for the ESP32 inference workflow
GitHub Copilot 16	Reported a serial port permission error while running the host SMILES sender against COM4.	Improved the Python script to catch serial open failures and print practical guidance about closing other serial monitors or processes
GitHub Copilot 17	Asked for a test pipeline that runs the dataset against the ESP32 logP model and produces charts	Added a resumable Python evaluation pipeline with subset selection modes, CSV summaries, and PDF plots
GitHub Copilot 18	Asked to reorganize the scripts for readability and maintainability.	Grouped imports, split constants into Paths and Hyperparameters sections, and reordered functions into Data, Model, Quantization and export, and Evaluation sections with separator comments
GitHub Copilot 19	Asked the ESP32 evaluator to retry failed molecules up to 100 times instead of moving on, and to treat device errors and timeouts as non-terminal so reruns retry them.	Added <code>MAX_RETRIES</code> constant, retry loop with serial buffer flush and re-PING between attempts, and changed <code>load_completed_indices</code> to only skip <code>ok</code> , <code>invalid_smiles</code> , and <code>max_retries_exceeded</code> results
GitHub Copilot 20	Reported that clamping persisted at 5.480893 after recalibration and asked how to eliminate it entirely.	Changed output type to float32 (TFLite inserts a DEQUANTIZE op; Dense layers still run int8) and updated the firmware to register <code>AddDequantize</code> , check for float32 output, and read <code>data.f[0]</code> directly.

System	Prompt	Usage
GitHub Copilot 21	Reported a boot loop after reflashing with the float32 output model.	Diagnosed peak arena overflow from the DEQUANTIZE op scratch buffers overrunning the heap during Invoke(); doubled kTensorArenaSize from 64 KB to 128 KB.
GitHub Copilot 22	Asked for a simple CLI script that takes a SMILES string and name, standardizes and canonicalizes it via the shared pre-processing module, generates a 2048-bit ECFP4 fingerprint, encodes the bit vector as a QR code, and saves it under the Results folder.	Added a Python CLI utility that writes QR images to Code/Results/qr_codes and installed the qrcode dependency for the active environment.
GitHub Copilot 23	Reported a QR generation failure and then asked to read the saved QR code back into a bit vector and throw an error if the round trip does not match.	Changed the QR payload to a compact hex-encoded fingerprint and added OpenCV-based QR decoding plus strict round-trip verification before reporting success.
GitHub Copilot 24	Asked for code to save a PDF graph of training/validation loss and MAE per epoch in Results/figures/history.pdf.	Added matplotlib code to plot and save training history after model training.
GitHub Copilot 25	Asked for a Python and RDKit figure showing an aspirin SMILES string being canonicalized, drawn as a molecule graph, passed through the neural-network training workflow, quantized, and packaged for ESP32 deployment.	Added a Python script that generates a figure and documented the canonicalization, training, quantization, and flashing stages.
GitHub Copilot 26	Asked for a simple RDKit script named molecule_picture.py that renders a specified molecule and saves it under Code/Results/molecules/worst_case.png.	Added a standalone Python script that canonicalizes the provided SMILES, renders the molecule with RDKit, creates the output directory if needed, and saves the PNG image to the requested location.

Declaration of Authorship

I hereby confirm that I have personally and independently prepared the present work and have not used any sources or aids other than those specified. All passages taken verbatim or in substance from other sources are identified as such. The drawings, illustrations and tables in this work are created by me or have been provided with an appropriate source reference. This work has not been submitted by me to any other university in the same or similar form for the acquisition of an academic degree.

Frankfurt, March 30, 2026.....

Rubin Chempananickal James